# Trabajo de Consulta: Funciones Anónimas

## Materia: Programación Funcional

Trabajo **individual** — Entrega: **semana 5**
Lenguajes: Scala • Java • Python

## Objetivos de aprendizaje

Al finalizar este trabajo, serás capaz de:

1. **Reconocer** dónde se usa la programación funcional en sistemas reales.
2. **Explicar** qué es una función anónima y por qué es la pieza clave de las operaciones con listas.
3. **Identificar** los tipos de problemas que se resuelven con funciones anónimas.
4. **Investigar** nuevas operaciones sobre colecciones y aplicarlas en la solución de problemas.
5. **Comparar** cómo Scala, Java y Python implementan las funciones anónimas y **argumentar**, con evidencia de tu propio código, por qué son importantes.

## Descripción

En clase trabajamos **operaciones con listas**. Cada vez que escribimos `lista.filter(p =>p.precio >10)`, la expresión `p =>p.precio >10` es una **función anónima**. En este trabajo vas a entender esa pieza a fondo, a reconocer en qué tipos de problemas se aplica y a comprobar, con ejemplos, por qué es tan útil.

El trabajo sigue una secuencia; cada parte usa lo que construiste en la anterior:

| **1. Explorar** → | **2. Entender** → | **3. Aplicar** → | **4. Ampliar** → | **5. Comparar** |
|---|---|---|---|---|
| casos reales | el concepto | tipos de problemas | operaciones nuevas | y concluir |

Todo se documenta en un repositorio de GitHub, con el archivo `README.md` como documento principal.

**Importante**

Cada tema se desarrolla **una sola vez**, en la parte que le corresponde. Si necesitas referirte a algo ya explicado, enlaza esa sección del README en lugar de repetirlo.

## Parte 1 – Explorar: la programación funcional en el mundo real

Antes de programar, investiga **dónde y para qué** se usa la programación funcional fuera del aula.

- Busca **2 casos reales** (empresas, productos o proyectos *open source*) que usen programación funcional o funciones anónimas. Algunos puntos de partida: Apache Spark (escrito en Scala), los Streams de Java en sistemas empresariales, pandas en análisis de datos con Python, o empresas que han publicado por qué adoptaron Scala.
- Para cada caso indica: qué problema resuelven, qué lenguaje o herramienta usan, qué ventaja les da el enfoque funcional y la fuente consultada (artículo, blog técnico, documentación o repositorio).
- Si encuentras un fragmento de código real con funciones anónimas (por ejemplo, en un repositorio de GitHub), inclúyelo y explica qué hace.

# Parte 2 – Entender: ¿qué es una función anónima?

Responde con tus palabras y apoya cada respuesta con un fragmento corto de código:

- ¿Qué es una función anónima y de dónde viene el nombre *lambda*?
- ¿En qué se diferencia de una función nombrada? Escribe la misma lógica de ambas formas.
- ¿Qué significa que las funciones sean *ciudadanos de primera clase*? ¿Por qué eso permite pasar una lambda a `map` o `filter`?
- ¿Qué es un *closure*? Muestra una lambda que use una variable definida fuera de ella.

# Parte 3 – Aplicar: ¿en qué problemas sirven las funciones anónimas?

Esta es la parte central del trabajo. Vas a identificar los **tipos de problemas** que se resuelven con funciones anónimas y a resolver cada uno con lambdas en Scala, Java y Python. La comparación entre los lenguajes se hace en la Parte 5.

## 3.1 Catálogo de tipos de problemas

Cada operación vista en clase resuelve un tipo de problema. Documenta al menos **4 de los 7 tipos** de la siguiente tabla:

| Tipo de problema | Pregunta que responde | Operaciones típicas |
|---|---|---|
| Selección | ¿Qué elementos cumplen una condición? | `filter` |
| Transformación | ¿Cómo convierto cada elemento en otro valor? | `map` |
| Validación | ¿Existe alguno que cumpla?, ¿todos cumplen? | `exists`, `forall` |
| Conteo | ¿Cuántos elementos cumplen una condición? | `count` |
| Extremos | ¿Cuál es el mejor o el peor según un criterio? | `maxBy`, `minBy` |
| Aplanamiento | ¿Cómo junto en una sola lista los resultados de cada elemento? | `flatMap` |
| Acumulación | ¿Qué valor resume la lista?, ¿cómo evoluciona paso a paso? | `foldLeft`, `scanLeft` |

Para **cada** tipo de problema documenta:

1. **Situación real:** un ejemplo concreto donde aparece ese problema (por ejemplo, *"un banco necesita detectar las transacciones mayores a $1 000"*).
2. **Solución con funciones anónimas** en los tres lenguajes:
   - **Scala**, con las operaciones vistas en clase.
   - **Java**, con Streams.
   - **Python**, con `lambda` y las funciones equivalentes (por ejemplo, `map`, `filter`, `max` con `key` o `functools.reduce`).
3. **Salida** de cada versión, para comprobar que las tres dan el mismo resultado.

**Sugerencia**

Usa una misma lista de objetos para todos tus ejemplos (por ejemplo, una `case class Producto(nombre, categoria, precio, stock)` con 10 a 15 elementos). Así no repites datos y las comparaciones son más claras.

# Parte 4 – Ampliar: cuatro operaciones nuevas

Las operaciones vistas en clase (`map`, `filter`, `exists`, `forall`, `count`, `maxBy`, `minBy`, `flatMap`, `foldLeft` y `scanLeft`) son solo una parte de lo que ofrecen las colecciones. Investiga **4 operaciones diferentes** a estas que te parezcan útiles para resolver problemas.

## 4.1 Elige tus operaciones

Puedes elegir entre las siguientes o proponer otras que encuentres en la documentación de Scala:

| Familia | Operaciones sugeridas |
|---|---|
| Ordenar | `sortBy`, `sortWith` |
| Agrupar | `groupBy`, `groupMapReduce` |
| Dividir una lista | `partition`, `span`, `takeWhile`, `dropWhile` |
| Buscar | `find`, `indexWhere` |
| Combinar listas | `zip`, `zipWithIndex` |
| Recorrer por bloques | `grouped`, `sliding` |
| Depurar y transformar | `distinctBy`, `collect`, `reduce` |

### 4.2 Ficha de cada operación

Para cada una de tus 4 operaciones, elabora una ficha breve con:

1. **Qué hace**, en una o dos líneas, con tus palabras.
2. **Qué función anónima recibe** y qué debe devolver esa lambda (por ejemplo, `partition` recibe una lambda que devuelve `Boolean`). Si la operación no recibe una lambda (como `zip` o `grouped`), muestra cómo se encadena con otra que sí la reciba.
3. **Ejemplo en Scala** usando la misma lista de objetos de la Parte 3, con su salida.
4. **Problema que resuelve**: una situación real donde la usarías y a cuál tipo de problema de la sección 3.1 se parece (o si es un tipo nuevo).

### 4.3 Combinar operaciones

1. Escribe una pregunta sobre tu lista de objetos que necesite varios pasos para responderse.
2. Respóndela con una sola cadena de operaciones que use **dos** de tus operaciones nuevas y **una** vista en clase.
3. Explica qué hace cada paso de la cadena.

**Ejemplo:** *"¿Qué categorías tienen más de 50 unidades en stock, ordenadas de mayor a menor?"* se responde con `groupBy` (nueva) → `filter` (vista en clase) → `sortBy` (nueva).

## Parte 5 – Comparar y concluir

### 5.1 Scala, Java y Python frente a frente

Con base en tus soluciones de la Parte 3, completa la siguiente tabla:

| Criterio | Scala | Java | Python |
|---|---|---|---|
| Sintaxis de la lambda | | | |
| Cómo se declara el tipo | | | |
| Operaciones con listas disponibles | | | |
| Restricción del lenguaje | | | |
| Legibilidad (tu opinión) | | | |

Luego responde: ¿en qué lenguaje fue más natural resolver los problemas?, ¿en qué tipo de problema encontraste más diferencias entre lenguajes y por qué?

### 5.2 Conclusión

En uno o dos párrafos responde: ¿qué importancia tienen las funciones anónimas en la solución de problemas con listas? Relaciona tu respuesta con al menos uno de los casos reales que investigaste en la Parte 1.

## Entregables

1. **Repositorio público en GitHub**:

```
README.md        <- documento principal (partes 1 a 5)
scala/           <- soluciones de la Parte 3
java/            <- soluciones de la Parte 3
python/          <- soluciones de la Parte 3
operaciones/     <- 4 operaciones nuevas y combinación (Parte 4)
img/             <- capturas de ejecución y tablas
```

2. Código comentado, ejecutable y con instrucciones breves para correrlo.
3. Capturas de la ejecución de cada ejemplo.

## Criterios de evaluación

| Criterio | Peso | Qué se evalúa |
|---|---|---|
| Parte 1 – Casos reales | 10 % | Casos pertinentes, bien explicados y con fuentes verificables. |
| Parte 2 – Comprensión | 10 % | Conceptos correctos, explicados con tus palabras y con ejemplos propios. |
| Parte 3 – Tipos de problemas | 35 % | Situaciones reales pertinentes y soluciones correctas en los tres lenguajes. |
| Parte 4 – Operaciones nuevas | 15 % | Operaciones bien elegidas, fichas completas, ejemplos correctos y combinación bien explicada. |
| Parte 5 – Comparación y conclusión | 20 % | Comparación entre lenguajes basada en tus ejemplos y conclusión argumentada. |
| Documentación | 10 % | README ordenado y sin repeticiones, código comentado y referencias citadas. |

## Fecha y forma de entrega

En la **semana 5** entregarás la URL del repositorio y presentarás en clase (máximo 10 minutos) dos de tus tipos de problemas resueltos en los tres lenguajes y tu principal conclusión.

# Referencias sugeridas

- Documentación oficial de Scala: `https://docs.scala-lang.org`
- Expresiones lambda en Java: `https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html`
- Programación funcional en Python: `https://docs.python.org/3/howto/functional.html`
- Winitzki, S. *The Science of Functional Programming: A Tutorial, with Examples in Scala*: `https://github.com/winitzki/sofp/releases/download/v2.2/sofp.pdf`
- Libros, artículos y blogs técnicos sobre programación funcional.

Cita todas las fuentes que utilices, incluidas las herramientas de IA si las usas.